

# SQLALCHEMY NO PYTHON

Conceitos fundamentais, SQLAlchemy Core e ORM

**Material didático**

## 1 INTRODUÇÃO

O SQLAlchemy é uma biblioteca do Python utilizada para trabalhar com bancos de dados relacionais. Ele fornece recursos para estabelecer conexões, construir consultas SQL e, principalmente por meio do ORM (Object Relational Mapper), representar tabelas do banco de dados como classes e registros como objetos Python.

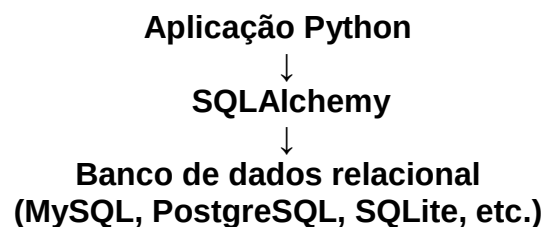
Em aplicações modernas, o SQLAlchemy pode atuar como uma camada entre o código Python e o sistema gerenciador de banco de dados. Dessa forma, o desenvolvedor consegue organizar melhor o acesso aos dados e reduzir a dependência de comandos SQL escritos diretamente em todas as partes da aplicação.

## 2 O QUE É O SQLALCHEMY?

O SQLAlchemy não é um banco de dados. Ele é uma biblioteca que permite que aplicações Python se comuniquem com bancos de dados relacionais e oferece diferentes formas de construir e executar operações sobre os dados.

Entre os bancos de dados que podem ser utilizados com SQLAlchemy estão MySQL, PostgreSQL, SQLite, Oracle e Microsoft SQL Server, entre outros.

Uma forma simples de visualizar seu papel é:



## 3 POR QUE UTILIZAR O SQLALCHEMY?

Em uma aplicação sem uma biblioteca de abstração, o programador pode precisar controlar diretamente conexões, comandos SQL e conversões entre dados do banco e estruturas utilizadas pelo programa. O SQLAlchemy oferece ferramentas para organizar essas responsabilidades.

Entre suas vantagens estão a organização do acesso aos dados, a possibilidade de utilizar SQL diretamente quando necessário, o suporte a ORM, a construção programática de consultas e a possibilidade de trabalhar com diferentes sistemas gerenciadores de banco de dados.

É importante destacar que utilizar SQLAlchemy não significa deixar de aprender SQL. Pelo contrário: conhecer SQL continua sendo fundamental para compreender consultas, relacionamentos, índices, constraints e desempenho.

## 4 SQLALCHEMY CORE E ORM

O SQLAlchemy possui dois conceitos importantes para o aprendizado: o SQLAlchemy Core e o ORM.

### 4.1 SQLAlchemy Core

O Core oferece uma camada de construção e execução de comandos SQL utilizando objetos e expressões Python. O desenvolvedor trabalha de maneira mais próxima do modelo relacional e do SQL.

```
stmt = select(usuarios).where(usuarios.c.nome == "João")
```

Nesse modelo, o programador ainda pensa em termos de tabelas, colunas e consultas, mas utiliza objetos Python para construir essas operações.

### 4.2 ORM – Object Relational Mapper

ORM significa Object Relational Mapper, ou Mapeamento Objeto-Relacional. A ideia é relacionar objetos e classes Python às estruturas existentes no banco de dados.

| Python |  | Banco de dados |
|--------|--|----------------|
|--------|--|----------------|

|                |   |                 |
|----------------|---|-----------------|
| Classe Usuario | ↔ | tabela usuarios |
| atributo nome  | ↔ | coluna nome     |
| objeto Usuario | ↔ | registro        |

Com ORM, uma tabela pode ser representada por uma classe Python. Os registros da tabela podem ser manipulados como objetos, tornando o código da aplicação mais próximo do modelo de programação orientada a objetos.

## 5 EXEMPLO DE UMA TABELA

Considere uma tabela chamada usuarios:

| usuarios |       |                 |
|----------|-------|-----------------|
| id       | nome  | email           |
| 1        | João  | joao@email.com  |
| 2        | Maria | maria@email.com |

Em uma aplicação Python, essa estrutura pode ser representada por uma classe Usuario. A classe funciona como o modelo utilizado pelo ORM para mapear a tabela.

## 6 CADASTRANDO UM REGISTRO

Depois de definir o modelo e configurar uma sessão, um novo usuário pode ser criado como um objeto Python:

```
usuario = Usuario(  
    nome="João",  
    email="joao@email.com"  
)  
  
session.add(usuario)  
session.commit()
```

O método `add()` coloca o objeto na sessão. O `commit()` confirma a transação, fazendo com que a alteração seja persistida no banco de dados.

## 7 CONSULTANDO DADOS

Uma consulta pode recuperar registros do banco. Em versões modernas do SQLAlchemy, recomenda-se utilizar a API baseada em `select()`:

```
stmt = select(Usuario).where(Usuario.nome == "João")  
usuarios = session.scalars(stmt).all()
```

O resultado pode ser percorrido como objetos Python, permitindo que o código da aplicação acesse atributos como `nome` e `email`.

## 8 ATUALIZANDO DADOS

Como o registro é representado por um objeto, seu atributo pode ser alterado e a sessão pode confirmar a mudança:

```
usuario.nome = "João Silva"  
session.commit()
```

O SQLAlchemy identifica a alteração e executa a operação necessária no banco durante a confirmação da transação.

## 9 EXCLUINDO DADOS

Um objeto também pode ser removido:

```
session.delete(usuario)  
session.commit()
```

A operação é efetivada no banco quando a transação é confirmada.

## 10 RELAÇÃO ENTRE PYTHON E SQL

Um dos pontos mais importantes para o estudante é compreender que o SQLAlchemy não elimina o banco de dados nem transforma SQL em algo desnecessário. Ele fornece uma camada de abstração para que operações de persistência possam ser expressas de diferentes maneiras.

**Python → SQLAlchemy → SQL → Banco de dados**

Por exemplo, uma operação de consulta feita com SQLAlchemy pode resultar, conceitualmente, em uma instrução SQL semelhante a:

```
SELECT id, nome, email
FROM usuarios
WHERE nome = 'João';
```

O SQL efetivamente gerado pode variar conforme a consulta, o banco utilizado e a forma como a instrução foi construída.

## 11 SQLALCHEMY E DIFERENTES BANCOS

Uma vantagem importante é a possibilidade de utilizar uma mesma biblioteca de acesso a dados com diferentes sistemas gerenciadores. Entretanto, isso não significa que todos os bancos sejam completamente idênticos. Recursos específicos, tipos de dados, funções e comportamentos podem variar.

**Entre exemplos comuns estão:**

- MySQL
- PostgreSQL
- SQLite
- Oracle
- Microsoft SQL Server

## 12 SQLALCHEMY NÃO É UM BANCO DE DADOS

Essa distinção deve ficar clara:

**SQLAlchemy ≠ MySQL**

**SQLAlchemy ≠ PostgreSQL**

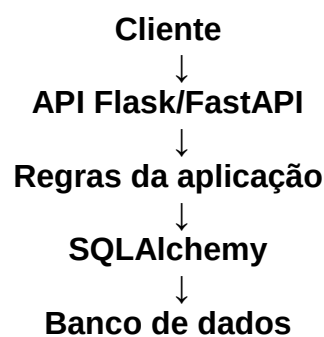
**SQLAlchemy ≠ SQLite**

MySQL, PostgreSQL e SQLite são sistemas utilizados para armazenar e gerenciar dados. SQLAlchemy é uma biblioteca utilizada pela aplicação Python para trabalhar com esses sistemas.

### 13 ONDE O SQLALCHEMY APARECE EM PROJETOS?

O SQLAlchemy é especialmente útil em aplicações web, APIs e sistemas que precisam persistir informações. Ele pode ser utilizado em conjunto com frameworks e ferramentas do ecossistema Python.

Um fluxo comum em uma API é:



Nesse cenário, uma requisição HTTP pode chegar à API, ser processada pela aplicação e resultar em uma consulta ou alteração no banco por meio do SQLAlchemy.

### 14 CONCEITOS QUE O ALUNO DEVE DOMINAR

Para utilizar SQLAlchemy com segurança e compreender seu funcionamento, é recomendável dominar previamente alguns conceitos:

- Python e orientação a objetos
- SQL básico
- SELECT, INSERT, UPDATE e DELETE
- Chaves primárias e estrangeiras
- Relacionamentos entre tabelas
- Constraints
- Transações
- Conceitos de banco de dados relacional

## 15 VISÃO GERAL DO FLUXO

De maneira resumida, o processo pode ser entendido da seguinte forma:

- 1) A aplicação Python é executada.
- 2) O SQLAlchemy é configurado.
- 3) A aplicação estabelece uma conexão com o banco.
- 4) Modelos podem representar as tabelas.
- 5) A aplicação cria, consulta, atualiza ou remove objetos.
- 6) O SQLAlchemy executa as operações necessárias.
- 7) O banco de dados persiste ou retorna os dados.

## 16 EXEMPLO CONCEITUAL COMPLETO

A seguir, um exemplo simplificado de como o ORM pode ser utilizado. O objetivo é demonstrar a ideia geral; a configuração completa de conexão e o mapeamento podem variar conforme a versão e o projeto.

```
from sqlalchemy import create_engine, String
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

engine = create_engine("sqlite:///escola.db")

class Base(DeclarativeBase):
    pass

class Usuario(Base):
    __tablename__ = "usuarios"

    id: Mapped[int] = mapped_column(primary_key=True)
    nome: Mapped[str] = mapped_column(String(100))
    email: Mapped[str] = mapped_column(String(150))
```

Nesse exemplo, a classe Usuario representa a tabela usuarios. Os atributos id, nome e email representam colunas. O banco utilizado é SQLite, escolhido apenas para manter o exemplo simples e fácil de executar.

## 17 RESUMO

SQLAlchemy é uma biblioteca Python para trabalhar com bancos de dados relacionais. Ela permite tanto construir consultas por meio de expressões Python quanto utilizar ORM para mapear tabelas para classes e registros para objetos.

O conceito central do ORM pode ser resumido assim:

|                        |   |                      |
|------------------------|---|----------------------|
| <b>Tabela do banco</b> | ↔ | <b>Classe Python</b> |
| <b>Coluna</b>          | ↔ | <b>Atributo</b>      |
| <b>Registro</b>        | ↔ | <b>Objeto</b>        |

Assim, SQLAlchemy pode ser entendido como uma camada de comunicação e abstração entre a aplicação Python e o banco de dados. Seu uso é particularmente relevante no desenvolvimento de APIs e aplicações que precisam armazenar e consultar dados de forma estruturada.

## 18 CONCLUSÃO

O aprendizado do SQLAlchemy representa uma evolução natural para quem já conhece Python e fundamentos de bancos de dados. Ao compreender SQL, modelagem relacional e orientação a objetos, o estudante consegue utilizar o SQLAlchemy para desenvolver aplicações mais organizadas e escaláveis.

A recomendação é aprender primeiro os fundamentos de SQL e banco de dados e, em seguida, compreender o SQLAlchemy Core e o ORM. Dessa maneira, o estudante não apenas aprende a utilizar uma ferramenta, mas entende o que acontece entre o código Python e o banco de dados.

## REFERÊNCIAS

**SQLALCHEMY.** SQLAlchemy Documentation. Disponível em: documentação oficial do projeto. Acesso em: 18 set. 2026.

**PYTHON SOFTWARE FOUNDATION.** Python Documentation. Disponível em: documentação oficial da linguagem Python. Acesso em: 18 set. 2026.